

Le noyau d'un item de liste

Force Tracker — 12/09/2026, ft-v1198. Dixieme document de la serie. Michel a pose le **test d'entree** en consigne permanente apres la sous-etape vide de la veille : « *verifie d'abord qu'il y a reellement au moins deux copies* » · « *extrait uniquement ce qui est strictement identique* » · « *toute divergence metier reste visible chez les appelants* ».

LE FAIT PRINCIPAL DE CETTE VERSION

Le plan annonçait DEUX divergences entre les trois sites, il n'y en a qu'UNE. Et il en deduisait un proprietaire **a parametre**, dimensionne pour un ecart qui n'existe pas.

=> *Le parametre n'a pas ete ecrit. Un parametre inutile deplace la divergence DANS le proprietaire, au lieu de la laisser visible chez les appelants — c'est exactement ce que la consigne interdit.*

Troisieme sous-etape d'affilee ou le perimetre ecrit dans le plan est faux. Le plan reste utile pour *ordonner* le travail ; il n'est plus une source pour les *chiffres*.

1. Le test d'entree, passe avant toute ligne

3 sites reels : la branche **favoris** et la branche **recents** de la construction de la liste « Mes aliments », et le **favori enregistre** quand on met une etoile. La regle du jour precedent tient : *une extraction exige au moins deux copies*, et on ne cree pas de proprietaire pour une forme unique.

2. Ce qui est extrait : 9 champs, mesures champ par champ

```
// AVANT — le meme noyau, recopie par TROIS sites

// _buildFoodQuickItems, branche FAVORIS
{name:f.name, kcal:f.kcal||0, ..., per100:f.per100||null, q:+f.q>0?f.q:0, u:f.u||null,
portionLabel:f.portionLabel||null, portionWeightG:...:0, fav:true}

// _buildFoodQuickItems, branche RECENTS
{name:e.name, kcal:e.kcal||0, ..., per100:e.per100||null, q:+e.q>0?+e.q:0, u:e.u||null,
portionLabel:e.portionLabel||null, portionWeightG:...:0,
origine:..., sourceId:..., etat:..., fav:false}

// toggleFavFood, le favori ENREGISTRE
{name:it.name, kcal:it.kcal||0, ..., per100:it.per100||null, q:+it.q>0?+it.q:0, u:it.u||null,
portionLabel:it.portionLabel||null, portionWeightG:...:null}

// APRES — 9 champs STRICTEMENT identiques dans un proprietaire ;
//          ce qui DIVERGE reste ecrit chez chaque appelant

function _itemListe(src){
  const s = src || {};
  return { name: s.name,
    kcal: s.kcal || 0, prot: s.prot || 0, carbs: s.carbs || 0, fat: s.fat || 0,
    per100: s.per100 || null,
    q: +s.q > 0 ? +s.q : 0,
    u: s.u || null,
    portionLabel: s.portionLabel || null };
}

Object.assign(_itemListe(f), {portionWeightG:...:0, fav:true}) // favoris
Object.assign(_itemListe(e), {portionWeightG:...:0, origine:..., fav:false}) // recents
Object.assign(_itemListe(it), {portionWeightG:...:null}) // le favori
```

Les 9 champs retenus sont ceux dont l'expression est **identique au caractere pres** aux trois sites — pas ceux qui se ressemblent: carbs, fat, kcal, name, per100, portionLabel, prot, q, u.

3. Le plan annonçait deux divergences, il n'y en a qu'une

champ	ce que le plan disait	ce que la MESURE dit
q	« 0 chez la liste, null chez le favori »	faux : +X.q>0?+X.q:0 aux trois sites — il ne diverge pas

champ	ce que le plan disait	ce que la MESURE dit
portionWeightG	« pareil »	la seule vraie divergence : 0 aux deux branches de la liste (2 fois), null au favori (1 fois)

POURQUOI CA CHANGE LA FORME DU PROPRIETAIRE

Le plan proposait `_itemListe(src, {vide:0})` contre `{vide:null}` — *un parametre nomme rend l'ecart visible dans le code au lieu de le cacher dans deux copies*. L'argument est bon **quand il y a deux ecarts**.

Avec **un seul**, le parametre n'achete rien : il fait entrer la divergence dans la signature du proprietaire, alors que l'ecrire chez l'appelant la laisse **lisible a l'endroit ou elle se decide**. **Le proprietaire ne prend donc qu'une source, sans option**.

4. Ce qui reste volontairement divergent

ce qui reste dehors	pourquoi
portionWeightG	replie sur 0 dans la liste, sur null dans le favori. Un 0 et un null ne se relisent pas pareil en aval ($+x>0$ les traite pareil, $x===null$ non) : unifier changerait le contenu enregistre des favoris. Decision produit n2, non tranchee.
fav	vaut true , false , et n'existe pas chez le troisieme. Trois etats, trois endroits.
origine / sourceId / etat	une seule copie (branche recents, 1 occurrence). On ne cree pas de proprietaire pour une forme unique — c'est la regle qui a fait ecarter la sous-etape precedente, et une mutation qui les y ferait entrer rougit .

Deux temoins de perimetre lisent le **corps** du proprietaire et exigent que ni `portionWeightG` ni `fav` n'y figurent. Deux gardes de ce document refusent de le produire si c'etait le cas, ou si le repli cessait d'etre **2 fois 0 et 1 fois null**.

5. La sonde : ouverte, pas crue — et juste cette fois

L'etiquette annonçait « *deja couvert* ». **Verifie en l'ouvrant** : elle appelle bien la production, avec les **deux** branches garnies (favoris ET recents), et conduit `toggleFavFood` **deux fois**, dont le cas ou tous les facultatifs sont absents.

APRES DEUX ETIQUETTES FAUSSES DE SUITE, CELLE-CI TIENT

La sous-etape 3-i annonçait « *instantane : couvert* » pour une sonde qui **recopiait la regle** au lieu d'appeler la production. La sous-etape suivante annonçait « *AUCUNE sonde* » alors qu'un temoin conduisait vraiment le code.

=> **Dans les deux sens, l'etiquette ne remplace pas l'ouverture du fichier**. Le cout de la verification est de deux minutes ; le cout de la croire est un critere de reussite qui ne peut plus rien detecter.

6. Deux pieges d'outillage, dont un nouveau

(1) Mon temoin de perimetre rougissait sur du code SAIN. Il listait les cles attendues du proprietaire : j'en avais ecrit **8 au lieu de 9**, en oubliant `name`. => **Un temoin de source se verifie d'abord contre le code sain** — s'il rougit la, c'est l'attendu qui est faux, pas le code. *Corrige avant toute mutation, sinon j'aurais cherche un bug qui n'existe pas*.

```
// LA MUTATION QUI NE MORDAIT PAS – et ce n'etait PAS un temoin manquant

// mutation : « per100 perdu »
"per100: s.per100 || null," -> "per100: null,"
// resultat : 0 rouge.

// Cause, trouvee en cherchant au lieu de conclure :
// ligne 1688 _srcRepriseQ -> per100: s.per100 || null,      <- ELLE frappait ICI
// ligne 2779 _itemListe -> per100: s.per100 || null,
// _srcRepriseQ porte le MEME motif et vient AVANT dans le fichier.
// Ses temoins vivent dans un AUTRE bloc, donc rien ne rougissait.

// Reancree sur deux lignes contigues propres a _itemListe :
// 2 rouges, exactement les deux temoins du pour-100 g.
```

UNE MUTATION MAL PLACEE EST INDISCERNABLE D'UN TEMOIN AVEUGLE

Les deux produisent le meme signal : **0 rouge**. Et la conclusion naturelle — « *ce code n'est pas couvert* » — est fausse dans un cas sur deux.

=> **Ancrer chaque mutation sur deux lignes contigues**, pas sur un motif d'une ligne. *Et ca resservira : plus on extrait de proprietaires, plus les motifs se ressemblent d'une fonction a l'autre — c'est un effet direct du travail en cours*.

7. Les mesures

preuve	resultat
Instantane, avant / apres	identique octet pour octet, sha256 7a52c37da93e17a3, diff vide
Controle negatif	11 mutations, TOUTES mordent — controle sain a 0 rouge sur 15 temoins, lance en premier
Passe complete	3620 / 3620 — total predit = total obtenu (3605 + 15)
Autres bancs	calculs 339/339 · muscles 241/241 · croises 50/50 · dates 9/9 · donnees : aucun trou nouveau
Source : proprietaire unique	4 occurrences (1 declaration + 3 appelants)
Source : divergence intacte	le repli vaut 2 fois 0 et 1 fois null

8. Etat du decoupage

Sous-etapes reelles restantes avant le hub	5 — sur 10 au plan, 4 livrees, 1 ecartee
Les 4 harmonisations	decisions produit , elles attendent Michel. Critere : est-ce que ca modifie ce qui est ECRIT dans le journal alimentaire ?
Le hub et la douane	apres, consigne explicite inchangee
Favoris perdus entre onglets, ecart 48,3 / 48	ouverts, non corriges
Historique, migrations	non touches

Document genere depuis le code servi — tous les decomptes cites sont recomptes a chaque generation (4 occurrences du proprietaire, 9 champs dans son corps, repli 2 fois 0 / 1 fois null, 1 copie de la provenance, 5 sous-etapes restantes). **Sept gardes refusent de produire si un fait tombe** — dont un qui verifie que `portionWeightG/fav` ne sont **pas** entres dans le proprietaire, un qui verifie que la divergence **n'a pas ete harmonisee**, et un qui verifie que le proprietaire **n'a pas gagne de parametre**. Source : `tools/gen_1biii_pdf.py`.